CS 24302 : COMPILER DESIGN LABORATORY

Lab Assignment 2 : Regular Expressions and use of Lex to generate DFA for Pattern Recognition

Scheduled : August 6 and 7, 2026

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

Objective :

- Write regular expressions (regexes) manually for tokens of interest.
- Get familiar with the structure of a Lex script.
- Rewrite regexes in the format of Lex and add actions to patterns as a lex script.
- Generate the lexical analyzer (also called a scanner) using Lex.
- Experiment with the generated scanner with sample inputs and understand its working.
- Augment the scanner to add more functionality.

P1. Recall the skills learnt in FLAT and use pen and paper to solve this problem.
Consider an alphabet Σ = {a, b, c} and the regular expression R, given by R = R1 | R2

$R1 = a^{+}b^{*}$ and $R2 = a^{*}c^{*}$

where the regex operators, "+", "*" and "|" have their usual meanings. The strings in the languages of R1 and R2, that is, L(R1) and L(R2) will be referred to as Token1 and Token2 respectively.

(a). Manually write the first 6 strings of L(R1) and L(R2) separately, in the order of increasing length.

(b). Determine if L(R1) ∩ L(R2) = Φ or not. If not empty, then write the 4 smallest strings that belong to L(R1) ∩ L(R2).

(c). Draw a DFA, say D, that recognizes L(R). Label each final state with the tokens (Token1 and /or Token2) it recognizes.

(d). Determine the behavior of D constructed by you for the input string, "abbcaaccbaca". The expected output is given below for reference. You need to explain each line of output with reasons.

D is supposed to be applied repeatedly till the entire string is exhausted and the output will comprise of all the sub-strings that are recognized as tokens, while symbols that do not match a pattern are printed as is.	Token 1 found Token 2 found Token 2 found bToken 2 found Token 1 found

P2(a). You are given a lex script in the file "**prob2.l**". Generate a scanner by performing the steps given in the handout, and give the executable a suitable name, such as "p2scan". Two input files, with names "p2inp1.txt", "p2inp2.txt", are given to you for testing. Run your generated scanner on both the inputs, and submit the outputs generated. Explain each line of the output with reasons.

(b). Modify "prob2.l" by interchanging the 2 lines as given below, leaving everything else unchanged.

prob2.l	prob2mod.l
a+b* {printf("Token 1 found \n");} a*c* {printf("Token 2 found \n");}	a*c* {printf("Token 2 found \n");} a+b* {printf("Token 1 found \n");}

Generate a scanner for the modified script "prob2mod.l". Generate outputs for the 2 input files of P2(a). Compare the outputs of P2(a) with P2(b) and explain their similarity and differences.

P3. Consider the two modified versions of "prob2.l" as given below. Notice carefully the small changes made in both the scripts as compared to the that in "prob2.l".

prob3a.l	prob3b.l
%% a+b* {printf("Token 1 found \n"); return 1;} a*c* {printf("Token 2 found \n"); return 2;} %% int main() { yylex(); }	%% a+b* {printf("Token 1 found \n"); return 1;} a*c* {printf("Token 2 found \n"); return 2;} %% int main() { while(yylex()); }

Now generate two scanners, from the lex scripts, and apply them to recognize the tokens for the 2 input files given in P2. Submit all the 4 output files and report your findings and explain the cause of the behavior of the scanner over these inputs.

P4. Change the lex scripts of P2 / P3, so that the generated parser generates the outputs against the inputs, shown below. Note that the scanner in this version, ignores white spaces (blank, tab or newline) and gives an error message when a symbol in the input does not match a regex.

Regex	Generated output of the Scanner
abbca c	T1 found : count = 1 Lexeme : abb T2 found : count = 1 Lexeme : c
bab	T1 found : count = 2 Lexeme : a T2 found : count = 2 Lexeme : c
cabc	Character b is either not a lexeme or not in Alphabet : count = 1 T1 found : count = 3 Lexeme : ab T2 found : count = 3 Lexeme : c T1 found : count = 4 Lexeme : ab T2 found : count = 4 Lexeme : c Statistics of tokens in input : |Token1| = 4; |Token2 | = 4 |All Tokens| = 8 Number of Characters in Input either not a lexeme or outside Alphabet : 1

P5. In the remaining time, if any, experiment with the functioning of the scanners generated by you by creating your own input files and / or modifying the lex script to change the behavior to your liking.

1. Handout to illustrate the process of generating a scanner from a lex script.

2. Structure of a lex script

3. Lex script, "prob2.l"

4. input files, "p2inp1.txt", "p2inp2.txt",

End of Laboratory 2 Experiment